问题与搜索方法实验课程设计

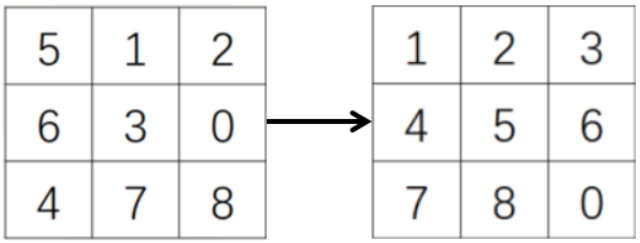
——八数码问题求解

一、实验目的

1. 掌握八数码问题的基本概念与状态表示方法；
2. 理解深度优先搜索（DFS）、广度优先搜索（BFS）、代价一致搜索（UCS）的原理与区别；
3. 实现三种搜索算法并比较其性能差异；
4. 强化工程实践能力，提升 C++面向对象编程能力与调试技巧。

二、问题描述

八数码问题：在 3×3 的棋盘上，摆有八个棋子，每个棋子上标有 1 至 8 的某一数字。棋盘中留有一个空格，空格用 0 来表示。空格周围的棋子可以移到空格中。给定一种初始布局（初始状态）和目标布局（如下图），找到一种移动方法，实现从初始布局到目标布局的转变。



动作：把与空格相邻的数字挪到空格所在位置。通过观察可以发现，如果空格位于棋盘中心，那么有“把 0 往上下左右挪动”四种可执行的动作；如果空格位于棋盘棱上，那么有三种可执行的动作；而如果空格位于棋盘的角上，就只有两种可执行动作。

后继状态：某个状态通过可执行动作形成的新的状态，例如对上面的初始状态，有下面 3 种后继状态：

5	1	2	5	1	0	5	1	2
6	3	8	6	3	2	6	0	3
4	7	0	4	7	8	4	7	8

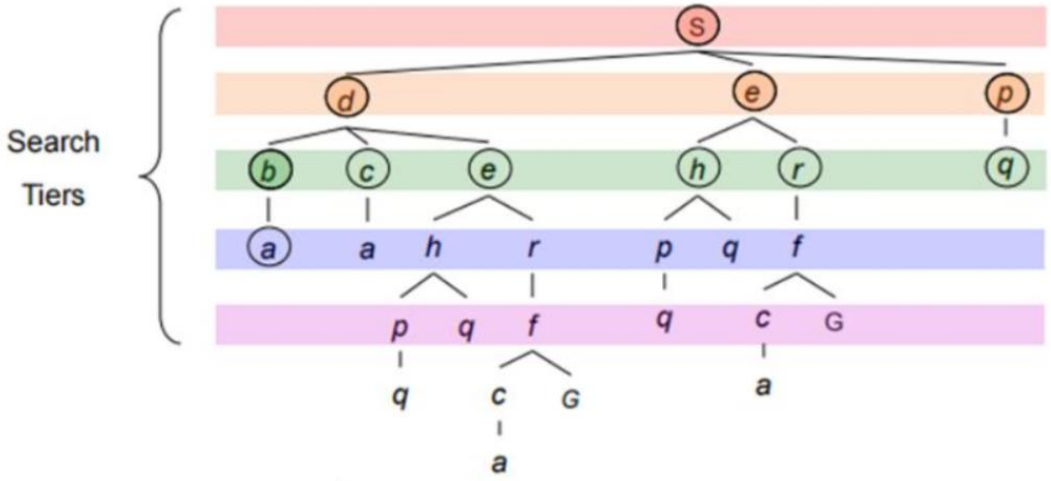
损失函数：记从初始状态开始，挪动的步数为损失函数值。我们希望找到从初始状态挪动最少步数达到期望状态的路径，也就是让损失函数最小。

要求补充所给程序中缺省的代码，实现上述功能，并在报告中记录设计、测试、分析等过程。

三、算法原理

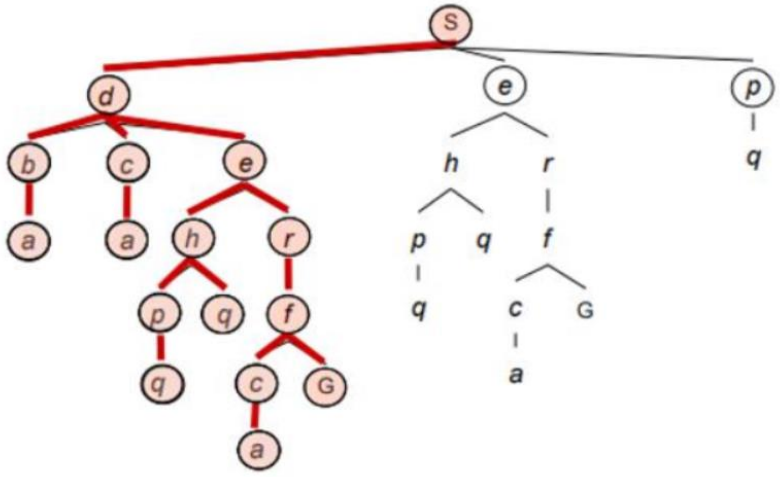
1. 广度优先搜索(BFS)

算法思想：优先探索最浅的节点。在八数码问题中就是首先将一个状态中空格的位置上、下、左、右移动完再继续子节点的搜索。



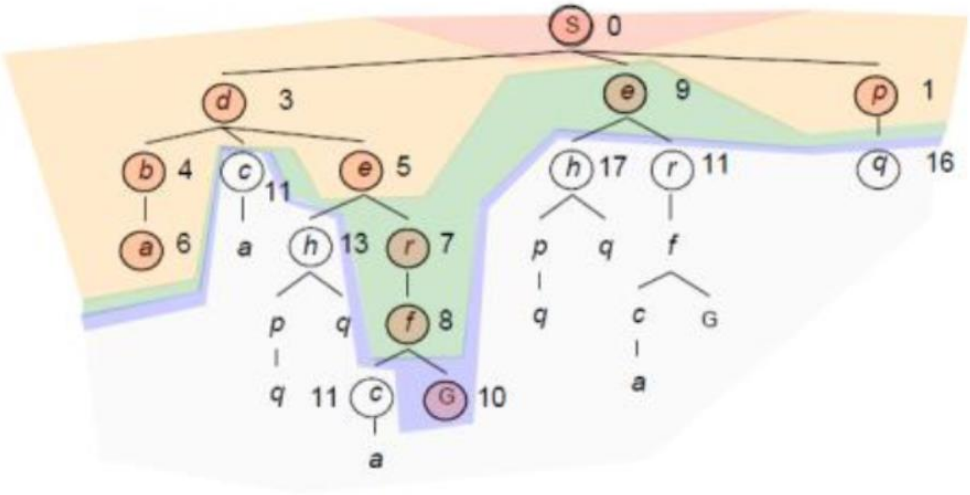
2. 深度优先搜索(DFS)

算法思想：优先探索最深的点。在八数码问题中先探索一个移动方向上的八数码状态，在实际问题中往往是一个有界深度优先搜索，因此搜索到边缘节点后再回溯进行其他方向的搜索。



3. 代价一致搜索(UCS)

算法思想：优先搜索代价最低的点。在八数码问题中，因为没有规定代价是什么，在处理的时候默认是路径的长度，也就是步长。因此，这个方法是在 BFS 的基础上进行的，里面的数据结构变成了优先队列，优先队列的元素定义了一个结构体 node，表示了当前八数码的状态以及路径代价（搜索步长）。



四、实验步骤

1. 环境配置：学习安装和使用 C++编译器和 IDE。
2. 代码实现：

- 确定好各个算法合适的数据类型和数据结构实现状态、动作、后继函数、损失函数等的表示。

  注意该问题存在以下核心设计要点：其一是要检查重复状态，由于问题中的

空格可以以多种方式出现重复状态，否则会导致程序陷入死循环；二是要进行内存管理，由于该问题的状态空间庞大，很容易超出机器的内存，应该尽量使用传引用，传指针的方式，做到不占用不必要的空间，此外 DFS 需要设置 MAX_DEPTH，否则可能因递归过深导致栈溢出；其三是路径回溯，所有算法均需通过前驱指针（pre）从目标结点反向回溯至初始结点。

- **广度优先搜索(BFS)**

推荐数据结构：queue（队列）

参考步骤：

（1） 初始化队列：将初始状态包装为结点（node）加入队列，并记录到历史列表（history）中。

（2） 循环取出队头结点：从队列头部取出一个结点进行扩展。

（3） 检查目标状态：

若当前结点状态是目标状态，回溯路径并输出解。

否则，生成当前结点的所有合法后继状态。

（4） 生成子结点：

对每个后继状态包装为新结点，设置前驱指针指向父结点。

（5） 查重与过滤：

若子结点状态已存在于历史记录中，跳过该子结点。

若子结点状态未访问过，将其加入队列尾部和历史记录。

（6） 重复扩展：

若队列非空，回到步骤 2 继续扩展。

若队列为空且未找到解，判定无解。

- **深度优先搜索(DFS)**

推荐数据结构：stack（栈）

参考步骤：

（1） 初始化栈：将初始状态包装为深度结点（初始深度为 0）压入栈中，并记录到历史列表（history）。

（2） 循环弹栈顶结点：从栈顶弹出一个结点进行扩展。

（3） 检查目标状态：

若当前结点状态是目标状态，回溯路径并输出解。

否则，生成当前结点的所有合法后继状态。

（4） 生成子结点：

对每个后继状态包装为新结点，设置前驱指针指向父结点，并更新深度（depth+1）。

若深度超过预设阈值（MAX_DEPTH），跳过该子结点。

（5） 排序与压栈：

对子结点按正确位置数（num_hit）降序排序（优化搜索顺序）。

逆序将子结点压入栈中（保证栈顶优先扩展最左分支）。

（6） 重复扩展：

若栈非空，回到步骤 2 继续扩展。

若栈为空且未找到解，判定无解。

- **代价一致搜索(UCS)**

推荐数据结构：priority queue（优先队列）

参考步骤：

（1） 初始化优先队列：将初始状态包装为代价结点（初始代价为 0）加入优先队列，并记录到历史列表（history）中。

（2） 循环取出最小代价结点：从队列头部取出当前代价最小的结点。

（3） 检查目标状态：

若当前结点状态是目标状态，回溯路径并输出解。

否则，生成当前结点的所有合法后继状态。

（4） 生成子结点：

对每个后继状态计算其累积代价（父结点代价+1）。

包装为新的代价结点，设置前驱指针指向父结点。

（5） 查重与更新：

若子结点状态已存在于历史记录中，比较新路径的累积代价：

新代价更低：更新历史记录中的代价，并将新结点加入优先队列。

代价相同或更高：跳过该子结点。

若子结点状态未访问过，将其加入优先队列和历史记录。

（6） 重复扩展：

若队列非空，回到步骤 2 继续扩展。

若队列为空且未找到解，判定无解。

3. 测试用例。
4. 性能分析。

对比实验结果表格示例参考：

算法	路径长度	历史状态数	时间(ms)
BFS			
DFS			
UCS			

请对结果进行解读分析。

五、思考题

1. UCS 在本实验中表现与 BFS 的结果比较有什么特点？为什么？
2. 如何改进 DFS 使其能找到最优解？
3. 当扩展节点时，采用不同的顺序（如先处理 up 还是 left）会对算法产生什么影响？

六、扩展任务（选做）

1. 实现 A*算法（启发式搜索）；
2. 支持 4×4（十五数码）乃至 N×N 的扩展版本（已提供 4×4 的测试用例可供使用）；
3. 如何检测无解情况？（提示：考虑逆序数奇偶性）；
4. 添加可视化移动过程，设计图形界面（建议使用 Qt）。